自主数据分析 Agent 技术方案

> NL2SQL 全链路 · 口径歧义澄清 · 多步编排与三技能 · SQL 与 Python 双安全

1. 问题与目标

业务取数的难点不在「把中文翻译成 SQL」，而在三处：

1. **口径同名异义**：销售额是含运费还是仅商品金额、净销售额是否扣除取消订单、月度趋势按哪个时间字段聚合——同一个词在不同部门指不同数；
2. **跨表关联复杂**：主表到维表的路径需要按业务语义选择，而不是任意可达路径；
3. **真实分析问题往往不是一次取数能回答**：需要取数、再计算、再查口径，多步依赖。

本项目围绕这三点实现完整链路：单轮的「理解 → 链接 → 生成 → 守卫 → 执行 → 解读」，以及外层多步编排。工程目标同样是可测量：每个环节都有对应的评测集与失败归因。

2. 系统架构

2.1 单轮链路（LangGraph 状态图）

```
用户问题
  └─ understand    拒答检测 + 口径与歧义识别
      ├─ refuse        变更类请求直接拒答
      └─ clarify       口径歧义 → 追问（评测模式下按默认口径记录）
          └─ link_schema    向量检索候选表与字段 + 同义词加权 + 外键桥接
              └─ gen_sql        生成 SQL
                  └─ guard          只读校验 / 表白名单 / 强制 LIMIT
                      └─ exec_sql       只读执行
                          ├─ retry（≤2 次）   携带错误信息回到 gen_sql
                          └─ interpret        口径化解读
                              └─ answer + 运行记录（trace）
```

任一环节失败按环节级重跑，不整链回退：错误分类为「生成错」时只重生成，保留已完成的链接结果，节省时延也避免链接结果被无关重试冲掉。

2.2 多步编排

外层三段式：`plan → execute_step → synthesize`。技能 3 个：

技能	职责	安全边界
`query_data`	复用的单轮取数链路	同 2.1 的守卫
`compute`	Python 沙箱计算	AST 静态检查 + 子进程隔离 + 超时
`search_doc`	口径文档检索	只读语料

`plan` 输出带依赖的步骤序列，`execute_step` 按依赖顺序执行并把上游结果注入下游输入，`synthesize` 汇总成结论。

3. 关键实现

3.1 口径体系：歧义在读 SQL 之前解决

口径配置在 `config/metrics.yaml`，每个歧义组含触发词、候选变体（各带关键词）、默认变体与澄清问句。`detect_ambiguity` 在触发词命中且问题与澄清文本都未指明变体时才追问；`resolve_variant` 按「澄清文本 → 问题自身 → 默认变体」的优先级确定实际口径。

现有 3 个歧义组：金额范围（支付总额 / 商品金额 / 已送达）、取消订单处理（是否扣除）、时间字段（下单 / 审批 / 送达）。

同名异义的处理必须前置——一旦 SQL 生成完再补救，成本是整条链路重跑。

3.2 Schema Linking：检索而非全量注入

候选集规模为百级（9 张表 + 52 个字段 + 23 条 few-shot），用嵌入检索召回候选表与字段，叠加同义词加权与外键桥接补齐关联路径。检索后端经显式对照（自部署 Qwen3-Embedding-4B 21/23 vs BGE-small 19/23）后接线，召回抽检 11/11。

外测时切换到全库 schema 注入模式并关闭专属口径层与索引，用于衡量「未经专门适配」的泛化能力——这是两套不同的评测模式，数字不能混用。

3.3 双层安全：SQL 守卫与 Python 沙箱

SQL 侧（`src/guard.py`）：先做原文正则拦截（写操作关键字），再用 sqlglot 解析，要求单条语句且为 `SELECT`，缺 `LIMIT` 自动补 1000。任一条不满足即拒绝执行，错误信息回填模型重试。

Python 侧（`src/sandbox.py`）：两层防护。

层	手段	拦截对象
静态	AST 遍历	`import` 语句、`__` 开头的名称与属性（禁内省与 `__builtins__` 逃逸）
运行时	独立子进程 + 白名单命名空间 + 超时	IO 与网络（命名空间内无 `open` 等接口）、死循环（超时杀进程）

模块自带 9 条自测用例（正常计算 4 条 / import 与内省 3 条 / IO 1 条 / 超时 1 条），全部通过。

3.4 变更类请求的边界处理

`detect_mutation` 识别「删掉 / 清空 / drop table / delete from / update ... set」等变更意图，命中即走拒答路径。这里有一个实测修正：早期正则对「删除率是多少」这类**名词化指标**误判，导致只读分析问题被拒答。修正方式是先排除 `删除(率|数|量|比例|次数)` 的名词用法，再匹配动词。修正后黄金集 8 条拒答用例仍 8/8 命中，误报消除。

4. 评测体系

三层评测，各自回答不同问题：

层	数据集	规模	回答什么
单轮黄金集	自建五类分层	72 条（执行 52 / 澄清 12 / 拒答 8）	主链路在受控条件下的准确率
多步任务集	自建五类	30 条	编排相对无规划的增益
跨库外测	BIRD Mini-Dev / CSpider / Archer / DataSpace	498 / 814 / 1012 / 60	未经适配时的泛化能力

黄金集为自建，参考 SQL 全量可执行；分层设计用于防止均值掩盖单类问题——澄清与拒答各占 12 与 8 条，权重足以让某个环节退化体现在总分上。

对照实验均采用同任务、同模型、只切一个变量（有无编排 / 有无口径层）。

5. 实验结果

5.1 单轮与迭代

指标	首轮基线	v0.1	v0.2（终版）
执行准确率（52 条）	41/52 = 78.8%	49/52 = 94.2%	52/52 = 100%
澄清触发（12 条）	12/12	12/12	12/12
拒答（8 条）	8/8	8/8	8/8
总体（72 条）	61/72	69/72	72/72

朴素基线（单次直出、无编排无口径层）39/52，编排加口径层净增益 +13 条 / +25.0pp。

5.2 多步编排

30 条五类任务：完成 30/30，最终答案 28/30（终版配置复跑区间 27–29）。无规划基线 12/30，净增益 +16 条 / +53.3pp。

迭代链为 14 → 23 → 25 → 28 → 29，每次提升都对应可定位的失败归因，记录在 `eval/CHANGELOG.md`。

5.3 跨库外测

基准	规模	结果
BIRD Mini-Dev（全库 schema 模式）	498 题	272/498 = 54.6%
CSpider（中文）	814 题	559/814 = 68.7%
Archer（中英双语，多跳反事实）	1012 题	238/1012 = 23.5%
DataSpace 公测（KDD Cup 2026）	60 题	A 单发 14/60 = 23.3%；B 取数+解读 12/60 = 20.0%

三个库的分数差异反映任务难度而非实现质量：BIRD 为中英混合真实库，CSpider 为中文语义解析，Archer 需多跳反事实推理，后者的 23.5% 说明「一次生成一条 SQL」的做法在复杂推理上存在结构性上限。

5.4 规模与引擎

真实分布放大 ×100 至 1,126.5 万行：全量聚合在 SQLite 上 6,615.6 ms，切 DuckDB 后 7.1 ms；双引擎结果一致性 49/52 = 94.2%，差异集中在方言函数——这条实证给出了「换引擎 + 转译层」的迁移路径与代价。

6. 工程实践

- **配置即资产**：口径、同义词、few-shot、提示词全部外置，迭代只改 `config/`，代码不动；这条纪律让每一轮改动的影响可归因；
- **可解释性**：每次运行落一份 trace（节点轨迹、时延、SQL、行数），评测归因与演示回放都基于它；
- **服务化外壳**：`service.py` 提供 `/v1/ask` 与 `/healthz`，`route=multi` 走多步编排；
- **单元测试**：66 条用例覆盖 SQL 守卫的 11 类拒绝分支、沙箱的静态与运行时拦截、口径识别与澄清分流。

7. 复现方式

```
python -m venv .venv && source .venv/Scripts/activate
pip install -r requirements.txt
python scripts/fetch_data.py && python scripts/build_db.py && python scripts/build_index.py
python app.py --mock "2018 年各州订单量排名？"     # 离线干跑（不调 LLM）
python eval/run_eval.py                            # 黄金集 72 条
python eval/run_multistep.py                       # 多步 30 条
python eval/run_external.py                        # BIRD Mini-Dev
python -m pytest                                   # 单元测试 66 条
```

离线干跑（`--mock`）会自动把生成模型与检索后端一并切到本地 mock 实现，不依赖任何外部服务；真实评测需在 `.env` 配置 Function Calling 端点。

8. 边界

- 主业务库为公开数据集（Olist），仓库不含数据本体，由脚本复原；口径体系为自建，不对应任何真实企业的指标定义；
- 公开基准均为本地自测口径，部分基准使用开发子集（BIRD Mini-Dev、CSpider 测试集、DataSpace 公测包），不对齐官方榜单；
- 视频模态数据在本环境不可解析，DataSpace 中含视频的 28 题已在报告中分层披露；
- 未做并发压测与多轮会话；服务化外壳为演示级单实例。